JOYMIE LACABA

QA Case Study · Testing a shipped app with no source and no access

Angeles, Pampanga, Philippines | GMT+8 | Open to Remote
joylacaba52@gmail.com | LinkedIn | GitHub | slashcal.com

WHAT THIS IS

A short account of how I test a mobile build I did not write and cannot see the source of — what I found, what I got wrong, and the check that caught it. Everything below was measured on a retail Android tablet against apps installed from Google Play. No SDK was added, no instrumentation, no cooperation from the developer.
I have led with the mistake rather than the finding, because the mistake is the part that decides whether the finding can be trusted.

THE CANDIDATE I REFUTED

A resource error that reproduced three times out of three — and was still not a defect

Coin Master Board Adventure · three cold launches

- Every run logged `No package ID 6a found for resource ID 0x6a0b0013` from the app's own process.
- `0x7f` is an app's own resources and `0x01` is the framework. `0x6a` is neither, and this build ships dynamic feature modules — so "a split that isn't loaded when the resource is asked for" fit the evidence.
- It reproduced in **all three runs**, which is normally what promotes a lead to a finding.
- Before writing it up I widened the search past the app's package. **Google Play Movies emitted the identical line with the identical resource ID.**
- Two unrelated applications cannot own the same app-local resource. `0x6a` belongs to a shared library on that device, and the miss is a device-side condition rather than anyone's bug.

Reproducing was not enough. The check that settled it — *does any other process on this device say the same thing?* — costs one command, and without it I would have sent a developer chasing something that was never theirs.

THE FINDING THAT HELD

A class the app loads at runtime that is not in the shipped build

Block Blast · corroborated on two independent channels

- **Channel one, the app's own log:** `ClassNotFoundException` for its Firebase reporting adapter, twelve times on every launch.
- **Channel two, the shipped artifact:** I pulled the installed APK and searched all **16 dex files**. The class is in none of them, while its package is present and populated and its sibling adapters — ThinkingData, AppsFlyer — both shipped.
- Either channel alone proves nothing. An exception could be handled; a missing class could be intentional. **Together they say the code asks for it and the build does not contain it, on every launch.**
- The shape — one class stripped, its neighbours kept, loaded through a factory — is what a shrinker does to a reflectively-loaded class. I wrote that up as a hypothesis with the two other explanations that fit the same evidence, not as a cause.
- The fix ships with a verification step against *the built artifact rather than the source*, because a keep rule that is present and not working looks identical to one that is absent.

THE ERROR I MADE, AND WHAT CAUGHT IT

I attributed another process's stalls to the app

- Looking for performance problems, I searched the log for main-thread stalls and found ten, the worst at 3.8 seconds.
- **Only six belonged to the app.** The others came from Android's own `system_server` and from a Lenovo pen service that happens to run on that tablet.
- My search matched on the text. The tooling I had already written binds every finding to the application's *process ID*, and it had been right the whole time — I just had not used it for that pass.
- The corrected finding is still real: six stalls in 116 seconds, worst 3,789ms, five of them carrying the same message identity, which narrows it to one repeated operation rather than five unrelated ones.

The reproduction steps in the report now end with the pid filter and a note that the same device emits identical-looking lines from processes that are not the app. That instruction exists because I needed it.

HOW I WRITE IT UP

- **Confidence stated per item.** “Measured”, “lead” and “refuted” are different words and get used as such.
- **Reproduction steps that work on someone else's machine**, including the traps.
- **What the report does not claim**, written down: not a cause, not a regression, not a pass. Absence of other findings is absence of looking.
- **Refuted candidates stay in the document.** A report that lists only what survived says nothing about how hard anything was looked at.

TOOLING

The runs behind this were driven by a test harness I built: it launches a build on real hardware, guarantees a process start the log can be bound to, streams the device log for the whole session, and files each run as a structured record with the app's own output attached. Two of the three items above came out of channels that need nothing from the developer — which is the point, since a stranger's binary is the normal case.